Chapter 3: Common programming concepts

Variables & Constants

'let' allows you do declare a variable which by default it's immutable: in order to make it mutable we need to add 'mut' while declaring it.

Constants on the other hand are so declared: 'const MAX_POINTS: u32 = 100_000;const MAX_POINTS: u32 = 100_000;'

Shadowing

As seen before in the guessing_game/ exercise we can declare a new variabile with the same name as a previuos variable, and the new variable shadows the previous.

We shadow a variable by using the same variable's name and repeating the use of the let keyword as follows:

```
fn main() {
    let x = 5;
    let x = x + 1;
    let x = x * 2;
    println!("The value of x is: {}", x);
}
```

This program first binds x to a value of 5 Then it shadows x by repeating let x =, taking the original value and adding 1 so the value of x is then 6. The third let statement also shadows x, multiplying the previous value by 2 to give x a final value of 12.

Shadowing is different than marking a variable as mut, because we'll get a compile-time error if we accidentally try to reassign to this variable without using the let keyword. By using let, we can perform a few transformations on a value but have the variable be immutable after those transformations have been completed.

The other difference between mut and shadowing is that because we're effectively creating a new variable when we use the let keyword again, we can change the type of the value but reuse the same name.

For example we might ask the user to input the amount of spaces they want between something, but we really need to store just a number:

```
let spaces = "     ";
let spaces = spaces.len();
```

This construct is allowed because the first spaces variable is a string type and the second spaces variable, which is a brand-new variable that happens to have the same name as the first one, is a number type.

However if we are trying to use 'mut' for that:

```
let mut spaces = "  ";
spaces = spaces.len();
```

We get the follwing error: "The error says we're not allowed to mutate a variable's type"

Data types

There are two data types subsets:

- scalar
- compound

In case when many types are possible, such as when we converted a String to a numeric type using parse in “Comparing the Guess to the Secret Number” we must add a type annotation, like this:

```
let guess: u32 = "42".parse().expect("Not a number!");
```

If we don’t add that the compiler will occour in to an error;

Scalar Types

A scalar type represents a single value. Rust has four primary scalar types: integers, floating-point numbers, Booleans, and characters. You may recog nize these from other programming languages.

Floating points

```
fn main() {
    let x = 2.0; // f64
    let y: f32 = 3.0; // f32
}
```

Boolean

```
fn main() {
    let t = true;
    let f: bool = false; // with explicit type annotation
}
```

Chars

```
fn main() {
    let c = 'z';
    let z = 'ℤ';
    let heart_eyed_cat = '😻';
}
```

Compound Types

Compound types can group multiple values into one type. Rust has two primitive compound types: tuples and arrays.

Tuple

A tuple is a general way of grouping together some number of other values with a variety of types into one compound type.

We create a tuple by writing a comma-separated list of values inside parentheses. Each position in the tuple has a type, and the types of the dif ferent values in the tuple don’t have to be the same. We’ve added optional type annotations in this example:

```
fn main() {
    let tup: (i32, f64, u8) = (500, 6.4, 1);
```

```
}
```

and we can then deconstruct this tuple by:

```
fn main() {
    let tup = (500, 6.4, 1);
    let (x, y, z) = tup;
    println!("The value of y is: {}", y);
}
```

In addition to destructuring through pattern matching, we can access a tuple element directly by using a period (.) followed by the index of the value we want to access. For example:

```
fn main() {
    let x: (i32, f64, u8) = (500, 6.4, 1);
    let five_hundred = x.0;
    let six_point_four = x.1;
    let one = x.2;
}
```

Array

```
fn main() {
    let a = [1, 2, 3, 4, 5];
}
```

Arrays are useful when you want your data allocated on the stack rather than the heap or when you want to ensure you always have a fixed number of elements.

An example of good array usage is to map store the month's names (static, always 12 elements).

In order to access a specific element we use:

```
fn main() {
    let a = [1, 2, 3, 4, 5];
    let first = a[0];
    let second = a[1];
}
```

When you attempt to access an element using indexing, Rust will check that the index you've specified is less than the array length. If the index is greater than the length, Rust will panic, which is the term Rust uses when a program exits with an error.

This is the first example of Rust's safety principles in action. In many low-level languages, this kind of check is not done, and when you provide an incorrect index, invalid memory can be accessed.

Functions

As for many others programming languages obviously the main() function is the program entry point.
'fn' allows you to declare a new function:

```
fn main() {
    println!("Hello, world!");
    another_function();
}
```

```
fn another_function() {
    println!("Another function.");
}
```

Rust uses snake case conventional style for functions and variables names: lowercase and underscores separate words. \

Note that we defined 'another_function' after the main function in the source code; we could have defined it before as well.
Rust doesn't care where you define your functions, only that they're defined somewhere.

Functions parameters

As for many others programming languages Rust also has parameters and arguments for its functions signatures; The following rewritten version of 'another_function' shows what parameters look like in Rust:

```
fn main() {
    another_function(5);
}

fn another_function(x: i32) {
    println!("The value of x is: {}", x);
}
```

The declaration of another_function has one parameter named x.
The type of x is specified as i32. When 5 is passed to another_function, the println! macro puts 5 where the pair of curly brackets were in the format string.

In functions signatures you MUST declare the type of each parameter which means that the compiler almost never needs you to use them elsewhere in the code to figure out what you mean.

Statements and Expressions in Function Bodies

Statements are instructions that perform some action and do not return a value.

Expressions evaluate to a resulting value.

For example:
Creating a variable and assigning a value to it with the 'let' keyword is a statement.

```
fn main() {
    let y = 6;
}
```

Statements do not return values. Therefore, you can't assign a let statement to another variable, as the following code tries to do: you'll get an error;

```
fn main() {
    let x = (let y = 6);
}
```

This is different from C where assignments returns the value of the assignment: in that language you can write 'x = y = 6' and both 'x' and 'y' contain the value 6.

Expressions evaluate to something and make up most of the rest of the code that you'll write in Rust.

Consider a simple math operation, such as '5 + 6', which is an expression that evaluates to the value '11'.

For example, calling a function is an expression, aswell as calling a macro or creating new scopes etc…

```
fn main() {
    let x = 5;

    let y = {
        let x = 3;
        x + 1
    };

    println!("The value of y is: {}", y);
}
```

The expression after 'y' is a block that, in this case, evaluates to 4.
That value gets bound to 'y' as part of the let statement before 'y'.

Note the line without a semicolon at the end, which is unlike most of the lines you've seen so far.
Expressions do not include ending semicolons. If you add a semicolon to the end of an expression, you turn it into a statement, which will then not return a value.

Functions with Return Values

Functions can return values to the code that calls them, in Rust we don't name return values but we declare their type after an arrow '->'.
The return value is the value of the final expression in the block of the body of a function.
You can also return early by using the 'return' keyword and specifying a value, here is an example:

```
fn five() -> i32 {
    5
}

fn main() {
    let x = five();
    println!("The value of x is: {}", x);
}
```

The 5 in five is the function's return value, which is why the return type is 'i32'.
We're using the return value of a function to initialize a variable, because the function five returns a 5, that line is the same as the following:

```
let x = 5;
```

The five function has no parameters and defines the type of the return value, but the body of the function is a lonely 5 with no semi colon because it's an expression whose value we want to return.

Here is another example:

```
fn main() {
    let x = plus_one(5);
    println!("The value of x is: {}", x);
}

fn plus_one(x: i32) -> i32 {
    x + 1
```

```
}
```

This works completely fine and print 'The value of x is: 6', but by adding a semicolon at the end of 'x+1':

```
fn main() {
    let x = plus_one(5);
    println!("The value of x is: {}", x);
}

fn plus_one(x: i32) -> i32 {
    x + 1;
}
```

We'll get an error: "note: 'expected type i32 found type ()'".
The definition of the function plus_one says that it will return an i32, but statements don't evaluate to a value, which is expressed by (), the empty tuple.
Therefore, nothing is returned, which contradicts the function definition and results in an error.

Control Flow

Introducing 'if' and loops expressions

if Expressions

Unlike languages such as JavaScript, Rust will not automatically try to convert non-Boolean types to a Boolean. You must be explicit and always provide if with a Boolean as its condition.

Handling multiple conditions with else if

```
fn main() {
    let number = 6;
    if number % 4 == 0 {
        println!("number is divisible by 4");
    } else if number % 3 == 0 {
        println!("number is divisible by 3");
    } else if number % 2 == 0 {
        println!("number is divisible by 2");
    } else {
        println!("number is not divisible by 4, 3, or 2");
    }
}
```

When this program executes, it checks each if expression in turn and executes the first body for which the condition holds true.
Note that even though 6 is divisible by 2, we don't see the output number is divisible by 2,nor do we see the number is not divisible by 4, 3, or 2 text from the else block. \ That's because Rust only executes the block for the first true condition, and once it finds one, it doesn't even check the rest.

Using if in a let statement

Because 'if' is an exxpression, we can use it on the right side of a let statement:

```
fn main() {
    let condition = true;
    let number = if condition {
        5
```

```
    } else {
        6
    };
    println!("The value of number is: {}", number);
}
```

We just have to make sure we provide the same type of values for each 'arm' otherwise we'll get an error like: "note: expected type {integer} found type &str"

Repetition with loops

Conditional loops: while

An example:

```
fn main() {
    let mut number = 3;
    while number != 0 {
        println!("{}!", number);
        number = number - 1;
    }
    println!("LIFTOFF!!!");
}
```

Looping through a collection: for

You can use a for loop and execute some code for each item in a collection.
A for loop looks like the code that follows:

```
fn main() {
    let a = [10, 20, 30, 40, 50];
    for element in a.iter() {
        println!("the value is: {}", element);
    }
}
```

Here's what the countdown would look like using a for loop and another method we've not yet talked about, rev, to reverse the range:

```
fn main() {
    for number in (1..4).rev() {
        println!("{}!", number);
    }
    println!("LIFTOFF!!!");
}
```